Multi-Agent Refactoring for Python

Fine-tuning a small local code model for guard-clause refactoring

Abdelrahman Magdy	Backend, integration and coordination
Abdullah Tarek	Agent workflow and verification
Abdelrahman Adel	Models and training
Shahd Abdelhay	Data, research and evaluation
Dania El-Walily	Frontend, report and presentation

The problem and our questions

- AI can rewrite code that looks cleaner but behaves differently
- So every proposal must be checked, and each check reported on its own
- We study one refactoring on one small Python function

Research questions

1. Do several coordinated roles beat one model call?
2. Does fine-tuning a small model help?
3. Is any gain worth its cost in time and calls?

What others found

Fowler	Defines guard clauses, the refactoring we use
LLM refactoring studies	Models often break behavior, so checks must be separate
Multi-agent tools	Shown with large closed models; we test a 0.5B local model
Self-repair	Corrections help only when the feedback is good
LoRA / PEFT	Lets a 4 GB GPU adapt a model
Tests and datasets	Tests can miss bugs; duplicates leak between splits

The task: flatten nested if statements

Before: depth 2

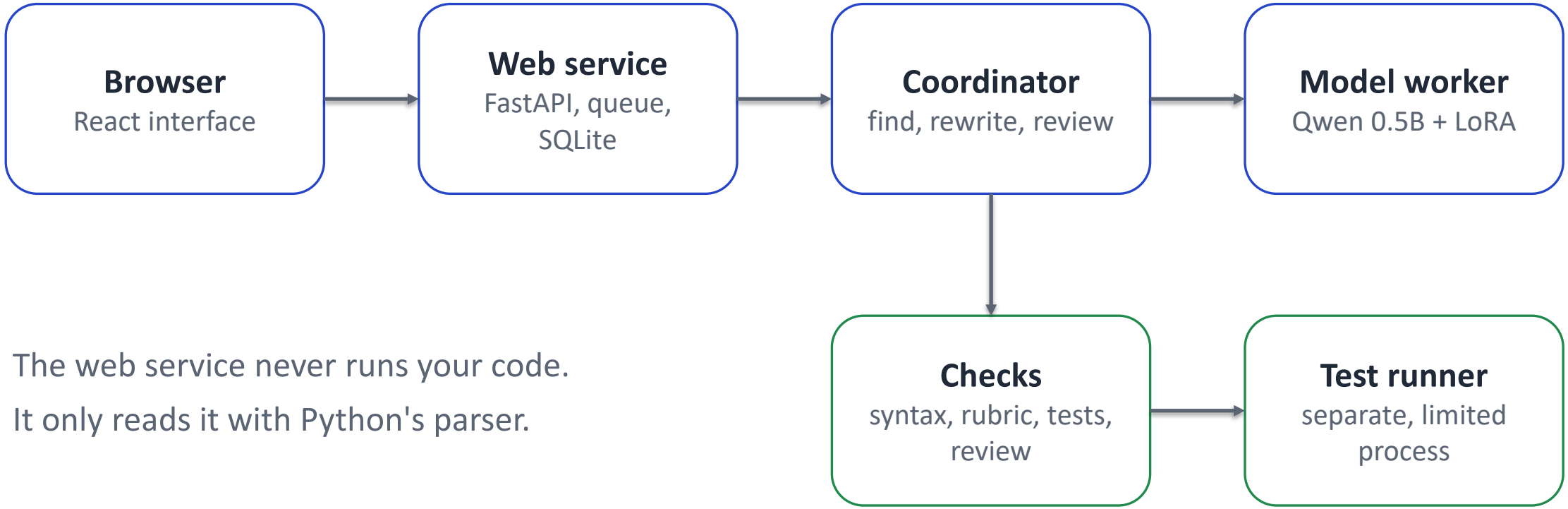
```
def shipping_cost(weight, express):
    """Return the shipping cost."""
    if weight > 0:
        if express:
            return weight * 3
        else:
            return weight * 2
    else:
        raise ValueError("weight must be positive")
```

After: depth 1

```
def shipping_cost(weight, express):
    """Return the shipping cost."""
    if weight <= 0:
        raise ValueError("weight must be positive")
    if express:
        return weight * 3
    return weight * 2
```

- One function, at most 60 lines; anything else gets “Nothing to change” with no model call

How the system fits together



The web service never runs your code.

It only reads it with Python's parser.

Two ways to run

Fast	**Rewrite** 1 model call
With review	**Find → Rewrite → Review** 3 calls, up to 5 with one retry

- Input outside the task stops before any model call
- Model output must match a strict format; one automatic repair at most

Four checks, never merged

Valid Python
Python's parser

Guard clauses
8 structure checks

Tests
trusted examples, isolated

Review
the model's opinion

The result is one of five:

Ready to use

Not safe to use

Not sure

Nothing to change

Something went wrong

- A missing check is never shown as a pass; the review never overrides a failed test
- Passing tests cover the tested inputs only, not every possible input

The model and the laptop

0.5B

Qwen2.5-Coder, Apache-2.0

4 GB

laptop GPU (RTX 2050)

~1 GB

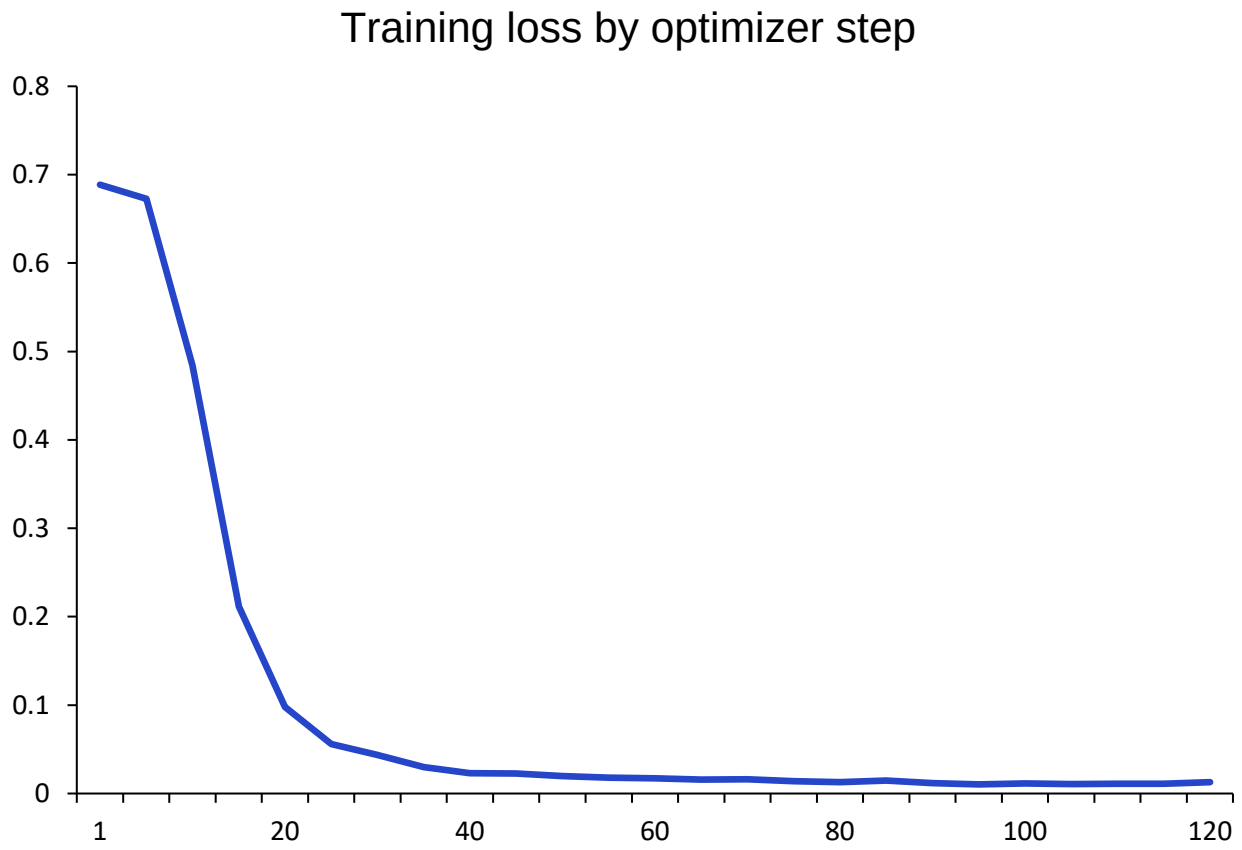
GPU memory in use

14-26

tokens per second

- The original model mostly copied the input back unchanged (14 of 24)
- Putting the code block first fixed the output format (24 of 24 parsed)
- So the open question was: can fine-tuning teach the actual rewrite?

Fine-tuning with LoRA



627

training functions

20.5 min

on a free Kaggle T4 GPU

0.69 → 0.01

training loss

69 vs 6

of 84 validation cases ready, with vs without the adapter

The dataset

799

functions

5,168

trusted tests

95.1%

of injected bugs caught

627 / 84 / 88

train / validation / test

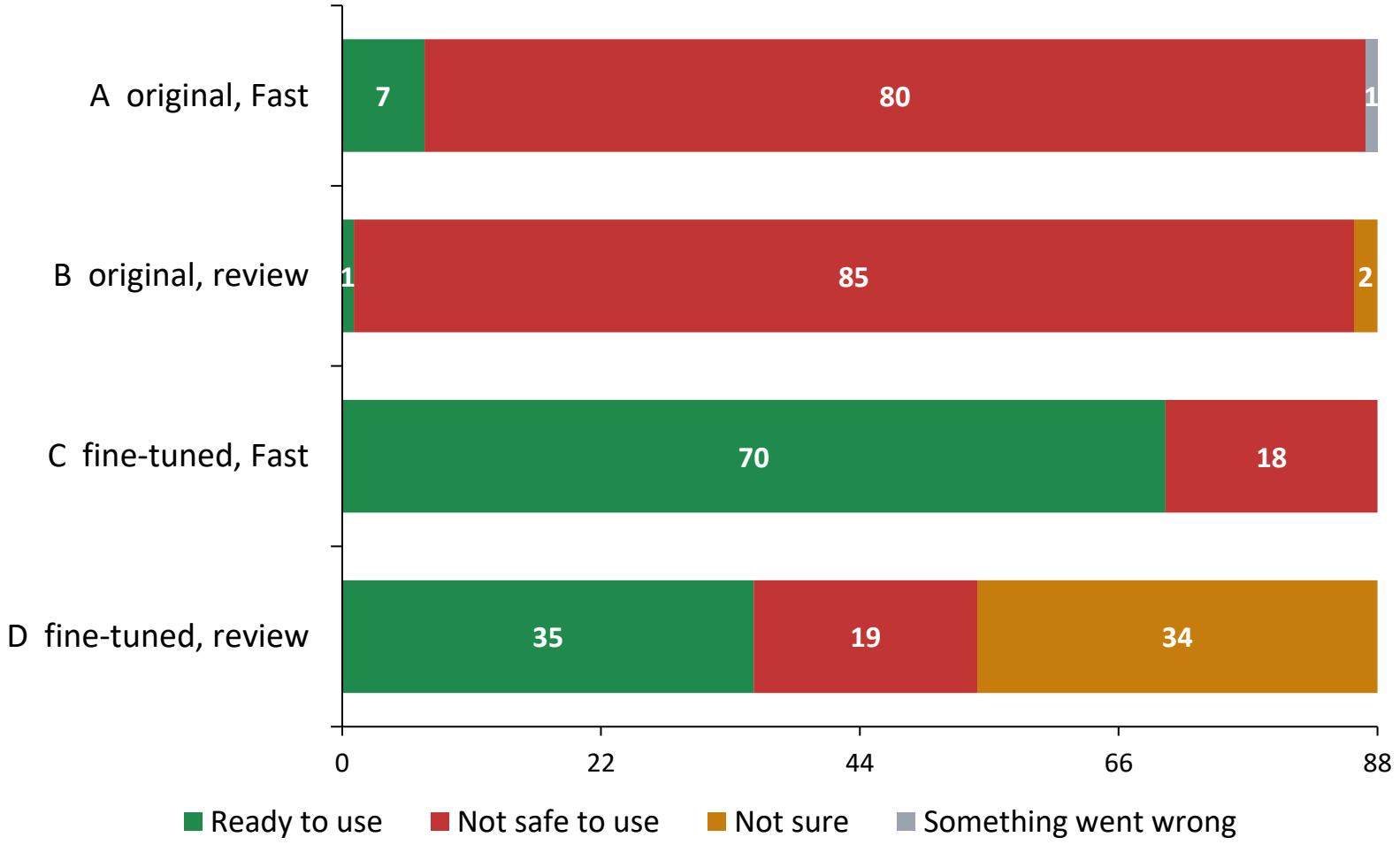
- Real code rarely has this pattern, so most functions come from team-written templates
- Splits keep whole template families apart, and duplicates were checked
- The 88 test cases were frozen before any final run

How we compared

	Original model	Fine-tuned model
Fast (1 call)	A	C
With review (3-5 calls)	B	D

- Same 88 test cases for all four
- Score: share marked “Ready to use”
- Targets fixed on validation first
- Paired tests: exact McNemar + Holm

Results on the 88 frozen test cases



79.5%

fine-tuned, Fast (C)

8.0%

original, Fast (A)

39.8%

fine-tuned, with review (D)

What the comparisons show

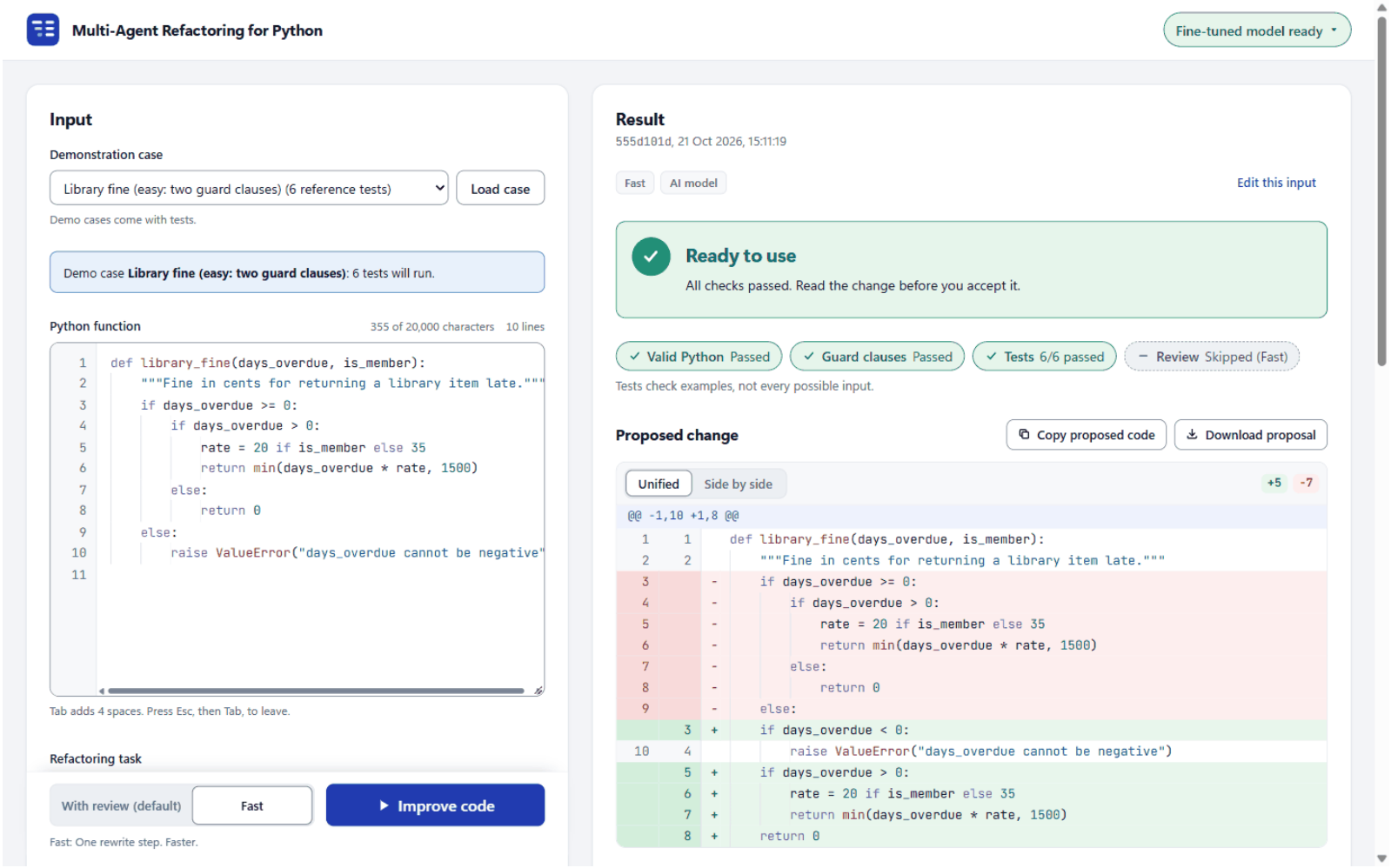
Comparison	Only first ready	Only second ready	p (Holm)
Fine-tuning, Fast (C vs A)	65	2	< 0.0001
Fine-tuning, with review (D vs B)	34	0	< 0.0001
Adding review, fine-tuned (D vs C)	1	36	< 0.0001
Adding review, original (B vs A)	0	6	0.031

- Fine-tuning helped a lot
- The review step lowered results: the untuned reviewer was often cut off
- 11 of 12 targets met; the with-review target was missed

Cost per case

Arm	Median time	Calls
A	3.0 s	1.0
C	7.7 s	1.0
B	16.0 s	4.9
D	17.9 s	3.4

The interface



- Verdict first, in plain words
- Four check chips
- The change as a diff
- Accept or reject only records a decision
- Your files are never changed

Live demonstration

1 A success — a demo case → Ready to use

2 Out of scope — an already flat function → Nothing to change

3 A failed check — invalid syntax, and a result without trusted tests

Limits and next steps

Limits

- One refactoring type, one small model
- Most data comes from templates
- Tests are examples, not proof
- The test runner is not a full sandbox
- Reproduced on one laptop only

Next steps

- A reliable reviewer (tuned, or not cut off)
- A second refactoring type
- More real code in the data
- A stronger isolation layer
- Try the 1.5B model

Built with AI coding assistance; each of us owns and explains our part.

Thank you

Questions?